

Security Review

Cantina Managed

POLYMARKET V1.2 UPGRADE

Cantina Managed review by:

Giovanni Di Siena, Lead Security Researcher

Red Swan, Lead Security Researcher



August 31, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
2.2	Trust assumptions	3
3	Findings	7
3.1	Low Risk	7
3.1.1	Zero taker-amount BUY orders make crossing checks vacuous	7
3.1.2	<code>00Reporter</code> delayed registration can strand requests when reused rules collide in UMA's global registration key	7
3.1.3	Zero-duration Chainlink candles collapse to a forced YES	8
3.1.4	Out-of-order bridge delivery can overcount neg-risk resolved conditions	8
3.2	Gas Optimization	9
3.2.1	Redundant leg length checks in <code>CombinatorialModule.prepareCondition()</code>	9
3.3	Informational	10
3.3.1	Consistent use of constants for shifting	10
3.3.2	<code>CtfRouter</code> upgrades do not enforce immutable dependency compatibility	10
3.3.3	Current paused resolutions in <code>resolutionPausedAt</code> will not persist on upgrade	10
3.3.4	Ambiguous variable naming	11
3.3.5	Inherited fractional neg-risk state can export synthetic Other as full YES	11
3.3.6	Routers emit canonical-looking success events after trusting registry-resolved module addresses	12



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

From Aug 25th to Sep 2nd the Cantina team conducted a review of [polymarket-v2](#) on commit hash [7656bb2a](#). The team identified a total of **12** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	4	0
Gas Optimizations	1	0	1
Informational	7	0	6
Total	12	4	7

2.1 Scope

The security review had the following components in scope for [polymarket-v2](#) on commit hash [7656bb2a](#):

```
├─ src
│   ├── bridge
│   │   ├── CcipBridge.sol
│   │   ├── abstract
│   │   │   └── BridgeBase.sol
│   │   └── interfaces
│   │       └── IBridge.sol
│   ├── collateral
│   │   ├── CollateralToken.sol
│   │   ├── abstract
│   │   │   └── CollateralErrors.sol
│   │   └── interfaces
│   │       └── ICollateralToken.sol
│   ├── exchange
│   │   └── Exchange.sol
│   ├── modules
│   │   ├── BinaryModule.sol
│   │   ├── CombinatorialModule.sol
│   │   ├── NegRiskModule.sol
│   │   └── migration
│   │       └── BaseMigrationMixin.sol
│   ├── oracle
│   │   ├── OracleAggregator.sol
│   │   └── abstract
│   │       ├── OracleAggregatorErrors.sol
│   │       ├── OracleAggregatorEvents.sol
│   │       └── OracleModuleBase.sol
│   └── routers
│       └── Router.sol
```

2.2 Trust assumptions

Exchange Operators and Exchange Administrators:

- Exchange operators are trusted not to intentionally choose settlement calldata that extracts value from users, but the exchange should still enforce signed order constraints against operator mistakes and edge-case calldata unless the behavior is explicitly accepted here.



- Off-chain order validation is not treated as an onchain safety boundary for fill rounding. If a non-zero maker fill can round the proportional taking amount down to zero despite the crossing condition, that outcome is either rejected onchain or accepted as relying on the trusted operator.
- Authorized exchange operators reject every taker or maker order whose `makerAmount` exceeds `type(uint248).max`; such an order is never submitted to `preapproveOrder` or `matchOrders` on either legacy venue or the v3 exchange.
- Authorized exchange operators do not intentionally choose non-zero fills that settle with zero counter-asset.
- The operator-supplied `takerFeeAmount` is not signed by each order maker; makers and takers rely on authorized operators and venue controls not to use this field to drain proceeds outside the expected order terms.

Makers and Signed Orders:

- For BUY orders, `makerAmount` is the collateral amount the buyer is willing to pay and `takerAmount` is the outcome-token amount requested. If `makerAmount > takerAmount`, the buyer is offering more collateral per outcome token than the token can ever redeem for at maximum payout.
- The protocol does not enforce a universal BUY price cap of one collateral unit per outcome token; above-max-payout BUY prices are treated as user-authorized or operator-screened terms.
- A maker's valid signature authorizes proportional partial fills, subject to the operator trust assumptions above; it does not by itself prove that every non-zero fill remains economically meaningful after integer rounding.
- Stale wallet owner or signer exposure in supported wallet-signature flows is known and accepted. Ownership or signer changes do not retroactively invalidate every previously valid order unless the relevant wallet, preapproval, or invalidation mechanism does so.

Protocol Governance, Migration, and Emergency Operations:

- Temporary differences between pUSD supply and liquid collateral currently held in the vault are accepted during legacy-position migration.
- Retained legacy claims are treated as sufficient eventual backing even before they resolve and their collateral is swept into the vault.
- Legacy migration exposure is expected to be small, and legacy claims are expected to resolve and be swept into the vault promptly.
- Migration operators and integrations do not call either `migratePositions(...)` overload for an unresolved legacy condition whose structured condition already has a stored result.
- `CollateralOnramp`, `CollateralOfframp`, and each deployed CTF collateral adapter are independent authorized wrap/unwrap pathways with separate per-asset pause state; pausing an asset on one contract is not expected to pause it on the others.
- Emergency operators pause the affected asset on every relevant ramp and CTF collateral adapter before treating that asset as quarantined; the system provides no single global wrap/unwrap pause.

Migration Creators:

- The migration-event creator calls `prepareMigrationEvent(legacyEventId)` before any result is accepted for a corresponding structured condition.
- Before calling `prepareMigrationEvent`, the creator verifies that every derived condition has an empty `result` and that the event has zero `resultsSum` and `conditionsResolved`. Registration stops for reconciliation if any state already exists.
- Once a migration event is prepared, no additional legacy questions or outcomes are added to that event; the prepared V2 arity remains the complete event arity.



Oracle and Reporter Operations:

- Oracle/request operators do not initialize, finalize, or otherwise route result reporting directly to the synthetic Other condition.
- Oracle operators and admins only configure `livenessWindow == 0` for requests where same-block finalization and no disputer/reporter-conflict challenge window are intentionally accepted.
- Oracle reporter module admins do not call `setAggregator` on a module with initialized, unresolved requests unless the replacement `OracleAggregator` has already reconstructed the same request configuration and registered the same reporter module for those events, or reporting is paused and the active requests are explicitly migrated. Reports routed through an inconsistent replacement aggregator are treated as privileged migration/configuration error, not a standalone permissionless vulnerability.
- Resolvers do not call `NegRiskModule.reportResult(conditionId, result)` for an event intended for legacy migration before `prepareMigrationEvent`; before registration, the module would treat the condition as native and store the result.
- Legacy NegRisk market creators include a fallback or “Other” question in every event so that an outcome exists even when no ordinary candidate wins or an unforeseen event invalidates the ordinary choices.
- The configured legacy market oracle always reports exactly one question as YES, using the fallback question when necessary, and never reports every question in an event as NO.
- For non-atomic native or migrated NegRisk events, result mass accrues to only one subcondition: one full-YES real condition or the synthetic Other condition, with all other real conditions resolving NO.
- `ATOMIC_NEGRISK` requests are used only for exhaustive real-outcome events where one listed condition index in `[0, arity)` must win. The synthetic Other condition at index `arity` is not a valid atomic winner.
- Partial NegRisk result mass split across subconditions, or any inherited partial real-condition numerator, is outside the accepted resolved-state model unless explicitly reconciled before continuing resolution.
- Chainlink reporter values are signed `int192` reports, and negative values are valid inputs. Future Chainlink-backed markets may use feeds such as derivative funding rates where the candle can move up or down across negative values; accepting a negative start or end price is not by itself a vulnerability unless the signed comparison or payout accounting is otherwise wrong.

Bridge Administrators and Relayers:

- Bridge administrators do not enable the relevant result routes, and relayers do not deliver results, until `prepareMigrationEvent` has completed on the destination. Constraining an operator alone is insufficient because `bridgeResult` is permissionless after route activation.
- Canonical bridge result messages may report the synthetic Other condition directly. That route is trusted to construct result payloads through the source-chain `bridgeResult()` path, including the module’s `getResultForBridge()` checks. For migrated NegRisk events, that source-side check must reject synthetic Other YES until every real migrated condition has a directly stored result.
- Any non-canonical `BRIDGE_ROLE` result path that bypasses source `bridgeResult()` / `getResultForBridge()`, or otherwise injects a synthetic Other payload, must be explicitly reviewed and reconcile the synthetic write with real-condition settlement before route activation.

Custody and Integration Boundaries:

- Users that leave tokens in modules do so at their own risk. Instead, users and integrations *should* use `Router.redeem`, `Router.convert`, `Router.merge`, or an equivalent contract that transfers and consumes positions atomically. Position balances left in a module after a transaction are aggregate, unattributed protocol custody and are not protected as deposits belonging to the original sender.



- This assumption does not exclude bugs that let module-held balances be misreported, aliased, manipulated, or over-consumed through obscure control flow, receiver behavior, proxy/delegate-call behavior, or ERC1155 accounting edge cases.
- Users and integrations do not transfer legacy CTF ERC-1155 positions directly to `CtfCollateralAdapter`, `NegRiskCtfCollateralAdapter`, or the legacy `NegRiskAdapter`; any position balance remaining on an adapter between transactions is unattributed, unprotected custody that may be consumed and paid to a later caller.

UMA:

- UMA is not the book of record for Polymarket's state.
- When the `OOReporterModule` delayed `createRequest` path is used, authorized Polymarket oracle/module operators register UMA with the latest canonical aggregator rule blob, including any rules recorded by `updateRequestRules` before the OO request was initialized. A stale or inconsistent operator-supplied rule blob is treated as privileged misconfiguration, not a standalone permissionless vulnerability.
- For incremental `NegRisk` events that rely on `OOReporterModule`, authorized oracle/module operators register one UMA request for every real condition index in the event before relying on OO settlement. An incomplete per-condition registration set is treated as privileged market-configuration error requiring release/preflight checks or operator repair via `createRequest`, not as a standalone permissionless protocol vulnerability.
- `OOReporterModule` intentionally inherits the shared owner-only upgrade guard without an onchain immutable-compatibility check for the constructor-set `OOReporter` address. Upgrade owners and release review are trusted to deploy replacement implementations with the same intended `OOReporter`, unless a reviewed migration deliberately changes that dependency and re-registers or reconciles every active UMA-backed request.



3 Findings

3.1 Low Risk

3.1.1 Zero taker-amount BUY orders make crossing checks vacuous

Severity: Low Risk.

Context: `Exchange.sol#L875`, `Exchange.sol#L1081`, `Exchange.sol#L1095-L1098`, `Exchange.sol#L1163-L1164`.

Description: For any valid positive `takerAmount` BUY fill, the old share-based status decrement is greater than or equal to the actual collateral spent. For positive `takerAmount`, this invariant holds; no obvious violation was found. The old share-based decrement remains conservative relative to collateral spent, so mixed pre/post status should not create overspend.

For `takerAmount == 0`, the invariant is undefined due to removal of the previous division-by-zero revert. BUY crossing checks become vacuous: the same-token SELL maker check becomes `< 0`, which is always false, and the complementary maker check also cannot fail because the zero taker amount collapses the right-hand side.

A malformed BUY order with `makerAmount = 50 USDC`, `takerAmount = 0 YES/NO`, and `side = BUY` can therefore be filled at an arbitrary price up to the full collateral budget, for example spending 50 USDC to receive 1 YES/NO outcome token, as long as the operator routes a matching maker order.

Recommendation: Explicitly reject `order.takerAmount == 0` in `_validateOrder()` if taker market orders are not intended.

Polymarket: Fixed in [PR 350](#).

Cantina Managed: Verified. Zero `takerAmount` orders are now rejected.

3.1.2 `OOReporter` delayed registration can strand requests when reused rules collide in UMA's global registration key

Severity: Low Risk.

Context: `OOReporterModule.sol#L248-L254`, `OOReporterModule.sol#L264-L265`, `OOReporterModule.sol#L305-L308`, `OracleAggregator.sol#L362-L364`.

Description: `OOReporterModule.createRequest()` registers a Polymarket request with UMA using a question-derived identifier and the caller-supplied `requestRules`. For binary and incremental neg-risk requests, the identifier is always `YES_OR_NO_QUERY`. UMA's managed `OOReporter` globally reserves each `(priceIdentifier, requestRules)` tuple and rejects the same tuple for a different `requestId`.

If an aggregator request is initialized with the OO module but empty init data, the aggregator request remains live while UMA registration is deferred. A later `createRequest()` using rules already registered by another binary-shaped market reverts inside UMA. The OO module then has no initialized request for that market, so `report()` cannot submit the UMA result. If the OO module is required for reporter quorum, normal resolution is blocked until privileged recovery.

Recommendation: Require `requestRules` to be unique per request before delayed registration, ideally by embedding the canonical `requestId` in UMA registration metadata. If rule semantics must remain unchanged, document and enforce a uniqueness preflight on `keccak256(abi.encode(identifier, requestRules))` before aggregator requests rely on OO quorum.

Polymarket: UMA is fixing this on their side in managed-oracle [PR 73](#) to allow sharing OO requests across duplicate Polymarket request IDs.



Cantina Managed: Acknowledged.

3.1.3 Zero-duration Chainlink candles collapse to a forced YES

Severity: Low Risk.

Context: ChainlinkReporterModule.sol#L164-L165, ChainlinkReporterModule.sol#L214-L217, ChainlinkReporterModule.sol#L310.

Description: ChainlinkReporterModule accepts candle configs with `duration == 0`. Initialization only checks that `startTimestamp + duration` is in the future, so a zero-duration candle with a future `startTimestamp` is valid.

At report time, the module computes `endTimestamp = startTimestamp + duration`. With `duration == 0`, both endpoint reads use the same `(feedId, timestamp)` datastore key. `startPrice` and `endPrice` are therefore identical, and the directional comparison `endPrice >= startPrice` always resolves to YES.

Because `report()` is permissionless after initialization, any caller can relay this deterministic YES result through the normal `OracleAggregator` flow. Once reporter quorum and liveness requirements are satisfied, the `BinaryModule` stores the result as terminal payout state.

This requires operator misconfiguration, but if zero-duration markets are not intentional, the reporter should reject them rather than silently turning them into guaranteed-YES markets.

Impact: A malformed Chainlink candle market can be accepted and later resolved to YES regardless of any price movement. NO holders can lose in a market that was configured as directional but had no actual observation interval.

Recommendation: Require positive duration during reporter initialization. If zero-duration products are intended, implement them as an explicit market mode with separate semantics instead of using the candle `endPrice >= startPrice` rule.

Polymarket: Fixed in PR 349. `initializeReporterModule()` now requires `duration > 0` alongside the existing future-window check, so the start and end reads can never be the same `DataStore` observation.

Cantina Managed: Verified.

3.1.4 Out-of-order bridge delivery can overcount neg-risk resolved conditions

Severity: Low Risk.

Context: @ commit `c0df0657`: NegRiskModule.sol#381,L386-L390,L403-L405.

Description: NegRiskModule uses `conditionsResolved[eventId]` as a count of stored results for a NegRisk event. The event arity counts only real conditions; the synthetic Other condition lives at index arity.

Bridge callers are allowed to store a direct result for the synthetic Other condition:

```
conditionIndex_ < conditionCount_  
|| (conditionIndex_ == conditionCount_ && hasAllRoles(msg.sender, BRIDGE_ROLE))
```

The finalizer then increments `conditionsResolved` for every stored result, including bridge-stored synthetic Other:



```

uint256 conditionsResolved_ = conditionsResolved[eventId_] + 1;
require(
    conditionsResolved_ <= conditionCount_ || resultsSum[eventId_] ==
        ↳ RESULT_DENOMINATOR,
    InvalidResults()
);

conditionsResolved[eventId_] = conditionsResolved_;

```

This means out-of-order result delivery can push the counter above the event arity. For an event with arity = 3, if the spoke receives Other YES first and later receives the three real NO results, the counter evolves as:

1. Other YES → conditionsResolved = 1.
2. Real 0 NO → conditionsResolved = 2.
3. Real 1 NO → conditionsResolved = 3.
4. Real 2 NO → conditionsResolved = 4.

The last write is accepted because `resultsSum[eventId] == RESULT_DENOMINATOR` after the synthetic YES. The module therefore reaches a state where `conditionsResolved` exceeds `conditionCount(eventId)` and hence the arity, even though the docs describe `conditionCount` as the number of real conditions and `conditionsResolved` as the number of stored results used for event resolution accounting. Valid bridge messages delivered in a different order can leave the destination module with an impossible-looking event counter.

Recommendation: Consider separating real-condition resolution accounting from stored synthetic bridge results.

Polymarket: Fixed in [PR 338](#). Synthetic Other no longer increments `conditionsResolved` (counter = directly stored real conditions, increment capped at arity) and the `resultsSum == RESULT_DENOMINATOR` overshoot escape is removed, so the counter cannot exceed arity in any delivery order. Pre-upgrade shims validated against all 14,018 live Polygon events.

Cantina Managed: Verified.

3.2 Gas Optimization

3.2.1 Redundant leg length checks in `CombinatorialModule.prepareCondition()`

Severity: Gas Optimization.

Context: [CombinatorialModule.sol#L398-L400](#), [CombinatorialModule.sol#L1005-L1008](#), [CombinatorialModule.sol#L1070-L1073](#).

Description: `CombinatorialModule.prepareCondition()` calls `_validateCanonical(_legs)` before `_storeLegsFromMemory(_legs)`. Both internal functions check `length > 0` and `length <= MAX_LEGS`, so the storage call repeats validation that already passed on this path.

Recommendation: Consider refactoring to avoid the repeated length checks.

Polymarket: Acknowledged. This is fine.

Cantina Managed: Acknowledged.



3.3 Informational

3.3.1 Consistent use of constants for shifting

Severity: Informational.

Context: `Exchange.sol#L491`, `Exchange.sol#L1241`, `Exchange.sol#L1287`, `Exchange.sol#L1305`.

Description: In recent changes to the codebase, constants have been preferred over numbers when shifting data in assembly (see commit `4b95887f`'s `ConditionIdLib`). In `Exchange.sol`, lines 491, 1241, 1287, and 1305 still use `8` to shift values.

There are several uses of numbers like this in `PositionManager.sol` as well. However, those are direct copies of Solady's library and probably have more value being idiomatic than cohesive to a style.

Recommendation: Create a new constant such as `_FILLED_FLAG_BITS` for the number `8` when shifting in the above locations.

Polymarket: Acknowledged.

Cantina Managed: Acknowledged.

3.3.2 `CtfRouter` upgrades do not enforce immutable dependency compatibility

Severity: Informational.

Context: `CtfRouter.sol#L66-L70`, `CtfRouter.sol#L180`.

Description: `CtfRouter` stores its `POSITION_MANAGER` and `COLLATERAL_TOKEN` references as implementation immutables, but `_authorizeUpgrade()` only checks `onlyOwner` and does not validate that a replacement implementation preserves those dependencies.

This means an owner-authorized upgrade can accidentally point the proxy at an implementation compiled with a different `PositionManager` or `CollateralToken`. Since `CtfRouter` uses those immutables to resolve modules and pre-transfer collateral before calling module operations, a dependency mismatch can break the router's core backing invariant or route user operations into a different protocol instance.

Recommendation: Match the module upgrade pattern by checking the new implementation's immutable dependencies in `_authorizeUpgrade()`.

Polymarket: Acknowledged. We will be managing upgrades with proper operational rigor.

Cantina Managed: Acknowledged.

3.3.3 Current paused resolutions in `resolutionPausedAt` will not persist on upgrade

Severity: Informational.

Context: `OracleModule.sol#L51`, `OracleModule.sol#L75`, `OracleModule.sol#L131-L132`, `OracleModule.sol#L139-L140`.

Description: One of the changes to the `OracleModule` contract was to change the `resolutionPausedAt` mapping to key on `EventId` instead of `ConditionId`. Storage slots for mapping values are computed from the slot the mapping is assigned and the key for the desired value. So where `EventId` and `ConditionId` do not match, the upgraded contract will not be able to see and enforce pauses for the old data `ConditionId`. This occurs precisely for `NegRisk` conditions that are of the non-zero index. Therefore, if such a condition was paused prior to upgrade, it will not be paused upon upgrading the contract unless the event is paused before the upgrade takes place.



Recommendation: Ensure all `NegRisk` condition pausing has their zero-index event paused prior to upgrading these contracts.

Polymarket: Acknowledged. Verified on-chain 2026-08-13: `pauseResolution` / `unpauseResolution` never successfully called on the deployed `Binary` / `NegRisk` modules, so the rekey strands nothing today. Will re-verify immediately before the upgrade executes.

Cantina Managed: Acknowledged.

3.3.4 Ambiguous variable naming

Severity: Informational.

Context: *(No context files were provided by the reviewer).*

Description: `BridgeBase` defines the immutable variable `RESOLUTION_CHAIN_ID` which is similarly named to `Ids`'s `ResolutionChain` enum. They however represent different concepts. In `BridgeBase`, it is the `chainid` of the network where results are finalized and must come from to be accepted. In `Ids` it is an index a module encodes into its position `ids` and expects its oracles to also index when sending results so that those positions may resolve. This shared nomenclature can be confusing as to what their intended purposes are.

Recommendation: Rename `RESOLUTION_CHAIN_ID` to something that describes its role as the location where results must be finalized. Candidates could include names like `RESOLUTION_ORIGIN_CHAIN_ID` or `RESULT_SOURCE_CHAIN_ID`.

Polymarket: Acknowledged. Fine with variable naming.

Cantina Managed: Acknowledged.

3.3.5 Inherited fractional neg-risk state can export synthetic Other as full YES

Severity: Informational.

Context: `NegRiskModule.sol#L296-L302`, `NegRiskModule.sol#L321-L323`, `NegRiskModule.sol#L460-L461`.

Description: The `NegRiskModule` implementation now treats neg-risk results as binary; however, if an existing v2 neg-risk event has a fractional real-condition result, current lazy derivation can overstate the synthetic `Other` condition.

The key issue is in `getResult()`:

```
if (conditionIndex_ == conditionCount_ && conditionsResolved[eventId_] ==  
    ↪ conditionCount_) {  
    result_ = new uint256[](2);  
    result_[0] = RESULT_DENOMINATOR;  
    return result_;  
}
```

This derives `Other = YES` solely from `conditionsResolved == arity` and does not require `resultsSum[eventId_] == 0`. This is safe for fresh post-upgrade results because `_storeResult()` now rejects fractional real payouts:

```
require(result0 == 0 || result0 == RESULT_DENOMINATOR, InvalidResults());
```

But it is not safe for inherited storage. The previously deployed `NegRiskModule` implementation allowed fractional real-condition payouts and accumulated them in `resultsSum`. For example, an event



may already have one real condition stored as `[500000, 500000]`, with `conditionsResolved = 1` and `resultsSum = 500000`.

After upgrade, if the remaining real conditions resolve NO, `conditionsResolved` can reach arity and `getResult(Other)` then returns `[1000000, 0]` even though a real condition already has non-zero YES mass. `getResultForBridge()` accepts this synthetic full-YES result because it checks only that each real condition has a stored two-element result:

```
for (uint256 i; i < conditionCount_; ++i) {
    if (result[eventId_.computeConditionId(i)].length != 2) {
        revert MigrationEventNotFullyResolved();
    }
}
```

It does not verify that real YES mass is zero before exporting synthetic Other YES. A canonical bridge export can therefore propagate an overpaying Other result to a destination chain.

Impact: A destination chain can receive and finalize `Other = YES` even though the source event contains fractional real YES mass. The fractional real payout and full synthetic Other payout are then both redeemable semantics for the same event, breaking the collateral accounting model. That said, impact is limited as it is understood that no live stored fractional neg-risk state currently exists.

Recommendation: Before upgrade or bridge deployment, enumerate existing neg-risk events and reconcile any event with nonzero partial `resultsSum` or fractional real-condition payouts. Additionally consider hardening the synthetic Other derivation in `getResult()`:

```
if (conditionIndex_ == conditionCount_ && conditionsResolved[eventId_] ==
    conditionCount_) {
+   require(resultsSum[eventId_] == 0, InvalidResults());
    result_ = new uint256[](2);
    result_[0] = RESULT_DENOMINATOR;
    return result_;
}
```

For `getResultForBridge()`, reject migrated or inherited events whose real stored results are not binary or whose aggregate YES mass is nonzero before exporting synthetic Other YES.

Polymarket: Acknowledged. Verified absent on-chain 2026-08-13: all 91,158 NegRisk Condition-Resolved stores on Polygon are exactly `[0,1e6]` or `[1e6,0]`, zero fractional results. Deployed `_storeResult()` still accepts fractional splits (resolver-gated). Will re-run as a pre-upgrade gate right before the `NegRiskModule` upgrade.

Cantina Managed: Acknowledged.

3.3.6 Routers emit canonical-looking success events after trusting registry-resolved module addresses

Severity: Informational.

Context: [CtfRouter.sol#L114](#).

Description: Both `Router` and `CtfRouter` derive `moduleAddr` from a caller-supplied `ConditionId`, `EventId`, or `PositionId`, then call the resolved module and emit a success event if that call returns. They do not independently verify that the resolved module is a canonical protocol module for the requested operation.

That means the emitted router events prove only that `PositionManager.moduleById(id)` returned an address and that the external call did not revert. They do not prove that the operation was handled by the real binary, neg-risk, or combinatorial implementation.



An invalid resolved module address, i.e. one that is controlled by the caller and implements the module interface, can make router events look like successful canonical operations even when the call was routed to a non-canonical implementation, potentially misleading indexers.

Recommendation: Consider hardening the routers with explicit module validation for the operations they expose.

Polymarket: Acknowledged. Indexers can verify the module ID by deriving it from the condition ID and reject appropriately.

Cantina Managed: Acknowledged.